AYUDA CODIGOS EXAMEN:

CONTROLLERS:

INDEX:

Listar reviews de un restaurante específico:

```
  async index (req, res) {
    try {
    const reviews = await Review.findAll(
      {
        where: { restaurantId: req.params.restaurantId },
                                //(req.user.id)
      })
    res.json(reviews)
  } catch (err) {
    res.status(500).send(err)
  }
},
```

CREATE:

```
async create (req, res) {
    const newReview = Review.build(req.body)
    newReview.restaurantId = req.params.restaurantId
                                //(lo que esté en la url)
    try {
      const review = await newReview.save()
      res.json(review)
    } catch (err) {
      res.status(500).send(err)
    }
  },
```

UPDATE:

```
async update (req, res) {
    try {
      await Review.update(req.body, { where: { id: req.params.reviewId
} })
      const updatedReview = await Review.findByPk(req.params.reviewId)
      res.json(updatedReview)
  } catch (err) {
      res.status(500).send(err)
  }
},
```

DESTROY:

```
  async destroy (req, res) {
    try {
      const result = await Review.destroy({ where: { id:
req.params.reviewId } })
      let message = ''
      if (result === 1) {
        message = 'Sucessfuly deleted review id.' + req.params.reviewId
      } else {
        message = 'Could not delete review.'
      }
      res.json(message)
    } catch (err) {
      res.status(500).send(err)
    }
  }
}
```

```
async markDefault(req, res) {
  const { shippingAddressId } = req.params
  const userId = req.user.id

  try {
    const [updated] = await ShippingAddress.update(
      { isDefault: Sequelize.literal(`CASE WHEN id =
${shippingAddressId} THEN true ELSE false END`) },
      { where: { userId } }
    )
    if (!updated) {
      return res.status(404).json({ message: 'Address not found or not
owned by user' })
    }

    const defaultAddress = await
ShippingAddress.findByPk(shippingAddressId)
    return res.status(200).json(defaultAddress)
  } catch (error) {
    console.error('Error en markDefault:', error)
    return res.status(500).json({ message: 'Internal server error' })
  }
}
```

MIDDLEWARES:

Tiene al menos un pedido en el restaurante que desea revisar (el usuario). ***count***

```
const userHasPlacedOrderInRestaurant = async (req, res, next) => {
  try {
    const userId = req.user.id
    const restaurantId = req.params.restaurantId //params = esta en url
    const ordersCount = await Order.count({where: {userId,
restaurantId}})

    if(ordersCount === 0){
      return res.status(409).json({error: 'No'})
    }

    next()
  } catch (err) {
    res.status(500).json({ error: err.message })
  }
}
```

No tiene ya una revisión publicada para ese restaurante. ***findOne***

```
const checkCustomerHasNotReviewed = async (req, res, next) => {
  try {
    const restaurantId = req.params.restaurantId
    const customerId = req.user.id
    const existingReview = await Review.findOne({
      where: { restaurantId, customerId }
    })
    if (existingReview) {
      return res.status(409).json({ message: 'You have already reviewed
this restaurant' })
    }
    next()
  } catch (error) {
    return res.status(500).json({ message: 'An error occurred while
checking reviews', error })
  }
}
```

Calcular media. ***this.getReviews()*** (sirve para obtener lista de todos)

```
async getAvgStars () {

      try {
        const reviews = await this.getReviews()
        if (!reviews.length) return

        const totalStars = reviews.reduce((acc, review) => acc +
review.stars, 0)
        return totalStars / reviews.length
      } catch (err) {
        return err
      }
    }
```

CHEKBELONGSTO y OWNERSHIP:

User no accede a ShippingAdress de otro: ***FindByPk(id)***

```
export const checkShippingAddressOwnership = async (req, res, next) =>
{
  // TODO
  try {
    const shippingAddressId = req.params.shippingAddressId
    const shippingaddress = await
ShippingAddress.findByPk(shippingAddressId)

    if (!shippingaddress) {
      return res.status(404).json({ message: 'Address not found' })
    }

    if (shippingaddress.userId !== req.user.id) {
      return res.status(403).json({ message: 'Forbidden' })
    }

    next()
  } catch (error) {
    console.error('Error in checkShippingAddressOwnership middleware:',
error)
    return res.status(500).json({ message: 'Internal server error' })
  }
}
```

Si dicen que x puede ser opcional:

```
if(!value){
  return Promise.resolve()
}
```

```
const checkScheduleBelongsToRestaurantOnUpdate = async (value, { req })
=> {
  if (!value) return Promise.resolve() // Permitir scheduleId vacío

  const schedule = await Schedule.findByPk(value)
  if (!schedule) {
    return Promise.reject(new Error('The scheduleId does not exist.'))
  }
  const product = await Product.findByPk(req.params.productId)//
  if (product.restaurantId !== schedule.restaurantId) {
    return Promise.reject(new Error('The scheduleId does not belong to
the restaurant of this product.'))
  }
  return Promise.resolve()
}
```

```
const showWithActiveProducts = async function (req, res) {
  res.status(500).send('To be implemented')
  try {
    const now = new Date().toTimeString().split(' ')[0] // 'HH:mm:ss'

    const restaurant = await
Restaurant.findByPk(req.params.restaurantId, {
      attributes: { exclude: ['userId'] },
      include: [
        {
          model: Product,
          as: 'products',
          required: false,
          include: [
            { model: ProductCategory, as: 'productCategory' },
            {
              model: Schedule,
              as: 'schedule',
              required: true,
              where: {
                startTime: { [Op.lte]: now },
                endTime: { [Op.gt]: now }
              }
            }
          ]
        },
        {
          model: RestaurantCategory,
          as: 'restaurantCategory'
        }
      ],
      order: [[{ model: Product, as: 'products' }, 'order', 'ASC']]
    })

    res.json(restaurant)
  } catch (err) {
    res.status(500).send(err)
  }
}
```

MIGRACIONES:

```
      userType: {
        allowNull: false,
        type: Sequelize.ENUM,
        values: [
          'customer',
          'owner'
        ],
        defaultValue: 'customer'
      },
      stars: {
        allowNull: false,
        type: Sequelize.INTEGER,
        validate: { // no penalizar en caso de ausencia
          min: 0,
          max: 5
        }
      },
      shippingCosts: {
        allowNull: false,
        defaultValue: 0.0,
        type: Sequelize.DOUBLE
      },
```

DUDAS

1. Si en los RF no aparece error 404, habría que añadirlo o hay que ceñirse a lo que el enunciado diga:
   Habría que ponerlo.
2. En el examen de ShippingAddress, en el Validation, porqué no se pone la FK y en update sólo se pone el isDefault
   Mirar RF3: PATCH sirve para modificar un único detalle, en este caso, isDefault. Lo demás se queda como estaba
3. Cómo sé los controllers que tengo que poner en el controller de la entidad. Si te dicen delete > destroy, put > update, post > create… pero cómo sé que tengo que hacer un: `indexByRestaurant, show, indexProductsByCategory`
   Ceñirse a lo que digan los RF.
4. **¿¿Cuándo se ponen las FK en Validation??**
   Básicamente es una validación más, habría que fijarse y ver si ponerla o no. Pero las FK suelen haberse chequeado previamente. Sin embargo, ponerla otra vez indica un código más seguro.

En definitiva, para qué se crea el index o hay que hacer los métodos CRUD en todos como esqueleto del controller, y si hiciera falta añadir alguno adicional pues se crearía?
Hay que ceñirse a lo que nos digan. En este caso nos pedían que listaramos las categorías de nuestro restaurante, por ende, sólo y digo sólo, habría que hacer el indexByRestaurant, los demás NO.

8. El orden de los middlewares en las rutas es relevante? Si es así, es el orden en el que aparecen en el RF?
Sí, no obstante, es algo subjetivo. Hay cosas lógicas como si te dicen *"Como propietario…"*, entonces habría que verificar primero que esté logueado y verificar el rol de *'owner'* antes de ver si existe la entidad o algo más.

9. Si te piden un PATCH como el siguiente, fijate en la ruta (en el Validation (update) sólo habría que poner un check para el atributo Default?):

Fíjate que la ruta asociada a este requisito es un `PATCH` `/shippingaddresses/:shippingAddressId/default`.

Sí, el PATCH solo se usa para actualizar unos valores específicos que te los indica el RF
10. Si te piden un PATCH, también habría que hacer el update en controller?
No, hay que ceñirse a lo que nos piden. En las soluciones de frontend viene el backend más completo de lo que nos piden a nosotros